

ScannerPoint: How a Full-Stack Web App Works

Oct 1, 2026 · @Aneesha

How to use these notes

ScannerPoint is three programs talking to each other: a **React** website in the browser, a **Spring Boot** server written in Java, and a **MySQL** database. Every section below explains one layer, using code that already exists in your repo as the example.

Read it in order the first time. Each section builds on the one before it. Afterwards, use it as a reference.

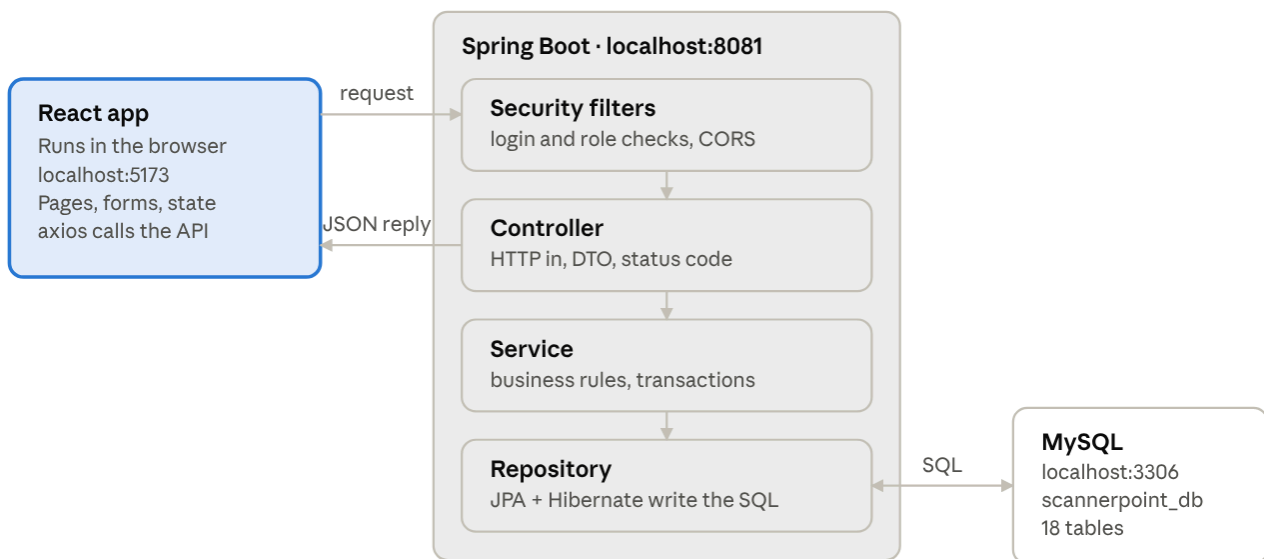
Layer	Technology in ScannerPoint	Where it lives	Runs on
Frontend (what users see)	React 19, Vite, React Router, axios	scannerpoint-web/ (deleted, to be rebuilt)	http://localhost:5173
Backend (the rules and logic)	Java 21, Spring Boot, Spring Security, Spring Data JPA	backend/src/main/java/ScannerPoint/...	http://localhost:8081
Database (the memory)	MySQL, database scannerpoint_db	Your MySQL server	localhost:3306
Build tools	Maven (backend), npm (frontend)	backend/pom.xml, package.json	Your terminal
Version control	Git and GitHub	.git/, the GitHub repo	Your machine and github.com

The backend is split into seven **modules**, one folder per business area: `user`, `customer`, `repair`, `inventory`, `billing`, `employee` and `config`. Each module (except `config`) has the same four layers inside it: `entity`, `repository`, `service`, `controller`, plus `dto`. Once you understand one module, you understand them all.

1. The big picture

A web app is a conversation: the browser **asks**, the server **answers**, and the database **remembers**. Nothing happens unless the browser sends a request first.

Every request passes through the backend before reaching MySQL



ScannerPoint architecture · 3 programs, 4 backend layers

A request enters at the top of the backend and moves down one layer at a time; only the repository layer ever talks to MySQL.

Client and server

- **Client** = the program that asks. Here it is the React app running inside the user's browser (Chrome, Safari).
- **Server** = the program that waits for questions and answers them. Here it is your Spring Boot app.
- **Database server** = a separate program that stores data on disk and answers questions written in SQL. Here it is MySQL.

The browser never talks to MySQL directly. Only the backend does. This is deliberate: the backend checks who you are and what you're allowed to do before touching the data. If the browser could reach the database, anyone could read the salary table.

Ports: three programs, one computer

During development all three programs run on your Mac. A **port** is a numbered door on a computer, so several programs can listen at once without mixing up their messages.

Program	Address	Set in
React dev server (Vite)	<code>localhost:5173</code>	<code>vite.config.js</code>

Program	Address	Set in
Spring Boot backend	localhost:8081	server.port=8081 in application.properties
MySQL	localhost:3306	MySQL's default; used in spring.datasource.url

`localhost` means "this same computer". When you deploy for real, these become real addresses such as `scannerpoint.lk` and `api.scannerpoint.lk`.

The request/response cycle

Every action a user takes follows the same loop:

1. The user clicks **Save** on the Add Customer form.
2. React gathers the form values and sends an **HTTP request** to `http://localhost:8081/api/customers`.
3. Spring Boot receives it, checks the user's login and role, validates the data, and runs your Java code.
4. Your Java code asks the database to **INSERT** a new row into the `customers` table.
5. MySQL saves it and tells Spring the new row's `id`.
6. Spring sends an **HTTP response** back: status `201 Created` and the new customer as **JSON**.
7. React receives the JSON and updates the screen. No page reload.

That is the whole of web development in seven steps. The rest of these notes zoom in on each step. Section 13 traces this exact example through your real code.

Stateless servers

HTTP is **stateless**: each request stands alone, and the server doesn't remember the previous one. That's why the frontend has to prove who it is on every request, by sending a login header each time (section 7).

2. HTTP and REST

HTTP is the language the browser and server use. REST is a set of conventions for organizing that language around **resources** (customers, vehicles, invoices).

Anatomy of a request

```
POST /api/customers HTTP/1.1
Host: localhost:8081
Content-Type: application/json
Authorization: Basic YWRtaW46YWRtaW4xMjM=

{"name": "Nimal Perera", "phone": "0771234567", "email": "nimal@mail.com"}
```

- **Method** (`POST`): what you want to do.
- **Path** (`/api/customers`): which resource.
- **Headers**: extra information as `Name: value` lines. `Content-Type` says the body is JSON. `Authorization` carries the login.
- **Body**: the data itself. `GET` requests usually have no body.

Anatomy of a response

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 7, "name": "Nimal Perera", "email": "nimal@mail.com", "phone": "0771234567", "address": null}
```

Methods (the verbs)

Method	Meaning	ScannerPoint example
<code>GET</code>	Read. Never changes data.	<code>GET /api/customers</code> : list all customers
<code>POST</code>	Create something new	<code>POST /api/customers</code> : add a customer
<code>PUT</code>	Replace a whole record	<code>PUT /api/employees/3</code> : update every field of employee 3
<code>PATCH</code>	Change part of a record	<code>PATCH /api/appointments/5/status?status=DONE</code>
<code>DELETE</code>	Remove	Not implemented anywhere in ScannerPoint yet

REST URL design

REST says: the URL names the **thing**, the method says the **action**. So it's `POST /api/customers`, not `/api/createCustomer`.

- Collection: `/api/customers`
- One item: `/api/customers/7`
- Items that belong to another item: `/api/vehicles/customer/7` (all vehicles of customer 7)

Path variables vs query parameters

Both put data in the URL, but they mean different things.

- **Path variable** = *which* resource. `/api/employees/3` → in Java: `@PathVariable Long id`.
- **Query parameter** = an option or filter, after the `?`. `/api/salary-payments?month=2026-09` → in Java: `@RequestParam String month`.
- **Body** = the full data for a create or update → in Java: `@RequestBody CustomerRequest request`.

Your `InventoryController.restockPart` takes its data as query parameters (`?partId=4&quantity=10`). That works, but sending a JSON body is more conventional for a `POST`.

JSON

JSON (JavaScript Object Notation) is the text format both sides use for data. Objects use `{ }`, lists use `[ ]`, and text goes in double quotes. Spring converts Java objects to JSON automatically (using a library called Jackson), and JavaScript reads JSON natively.

```
{ "id": 7, "name": "Nimal", "vehicles": [ { "licensePlate": "CAB-1234" } ],
  "active": true }
```

Status codes

The three-digit number tells the client what happened before it even reads the body.

Code	Name	When ScannerPoint should use it
200	OK	A successful GET, PUT or PATCH
201	Created	A successful POST (your controllers do this with <code>HttpStatus.CREATED</code>)
204	No Content	A successful DELETE
400	Bad Request	Validation failed (blank name, invalid email)
401	Unauthorized	Not logged in, or wrong password

Code	Name	When ScannerPoint should use it
403	Forbidden	Logged in, but your role isn't allowed (a mechanic opening payroll)
404	Not Found	<code>/api/employees/999</code> doesn't exist
409	Conflict	Duplicate: username already taken, NIC already exists
500	Internal Server Error	The server crashed. Should never happen on purpose

Rule of thumb: **4xx = the client did something wrong**, **5xx = the server did**. Right now your backend returns 500 for "customer not found", which is a known issue (section 6).

3. Databases

A relational database stores data in **tables** that are linked to each other by IDs. MySQL is the program that stores the tables and answers questions about them in **SQL**.

Tables, rows and columns

A table is like a spreadsheet with strict rules. Each **column** has a fixed name and type. Each **row** is one record.

`customers` table:

id	name	email	phone	address
1	Nimal Perera	nimal@mail.com	0771234567	Kandy
2	Sita Fernando	NULL	0719876543	Colombo 5

`NULL` means "no value". It is not the same as an empty string `""` or zero.

Keys

- **Primary key (PK):** a column that uniquely identifies each row. In ScannerPoint it's always `id`, and MySQL generates it automatically (`AUTO_INCREMENT`). Never reuse or change an id.
- **Foreign key (FK):** a column that stores *another table's* id, creating a link.
`vehicles.customer_id = 1` means "this vehicle belongs to customer 1".

- **Unique constraint:** the database refuses a second row with the same value.
`vehicles.license_plate`, `users.username` and `employees.nic` are unique.

Relationships

Type	Meaning	ScannerPoint example	How it's stored
One-to-many	One A has many B	One customer has many vehicles	FK <code>customer_id</code> on the <i>many</i> side (<code>vehicles</code>)
Many-to-one	The same link seen from B	Many job cards point at one vehicle	FK <code>vehicle_id</code> in <code>job_cards</code>
One-to-one	Exactly one each	An employee has at most one login account	FK <code>user_id</code> in <code>employees</code> , marked unique
Many-to-many	Many A, many B	A user can have several roles; a role belongs to many users	A third join table , <code>user_roles</code> (<code>user_id</code> , <code>role_id</code>)

Your schema

Hibernate creates these tables from your Java entity classes (section 4). These are the 18 tables in `scannerpoint_db`, grouped by module:

Module	Tables	Key links
user	<code>users</code> , <code>roles</code> , <code>user_roles</code>	<code>user_roles</code> joins users ↔ roles
customer	<code>customers</code> , <code>vehicles</code>	<code>vehicles.customer_id</code> → customers
repair	<code>appointments</code> , <code>job_cards</code> , <code>repair_tasks</code> , <code>inspections</code>	<code>appointments</code> → customer and vehicle; <code>job_cards</code> → vehicle and <code>mechanic_id</code> → users; <code>repair_tasks</code> → <code>job_cards</code> ; <code>inspections</code> → vehicle
inventory	<code>suppliers</code> , <code>spare_parts</code> , <code>inventory_transactions</code> , <code>part_usages</code>	<code>spare_parts</code> → supplier; <code>part_usages</code> → <code>job_card</code> and <code>spare_part</code>
billing	<code>invoices</code> , <code>invoice_items</code> , <code>payments</code>	<code>invoice_items</code> and <code>payments</code> → invoice. Note: <code>invoices.customer_id</code> and <code>job_card_id</code> are plain numbers, not real foreign keys

Module	Tables	Key links
employee	employees , salary_payments	employees → users (optional); salary_payments → employee, unique per (employee, month)

The billing note matters: because `invoices.customer_id` is not a real FK, MySQL will happily accept an invoice for customer 9999 who doesn't exist. Real FKs protect you from that. This is called **referential integrity**.

SQL in five statements

You rarely write SQL in ScannerPoint because Hibernate writes it for you. But you must be able to read it, because `spring.jpa.show-sql=true` prints every query in your console.

```
-- Create
INSERT INTO customers (name, phone, email) VALUES ('Nimal Perera',
'0771234567', 'nimal@mail.com');

-- Read
SELECT id, name, phone FROM customers WHERE name LIKE 'Nim%' ORDER BY name;

-- Update
UPDATE spare_parts SET quantity_in_stock = quantity_in_stock - 2 WHERE id =
4;

-- Delete
DELETE FROM vehicles WHERE id = 12;

-- Join: combine two tables through the foreign key
SELECT v.license_plate, c.name
FROM vehicles v
JOIN customers c ON v.customer_id = c.id;
```

The first four are called **CRUD**: Create, Read, Update, Delete. Almost every screen in ScannerPoint is a CRUD screen.

Transactions

A **transaction** groups several changes so that either all of them happen or none do. Dispensing a part should (1) reduce stock, (2) record the part usage, (3) log the inventory transaction. If step 2 fails after step 1, you'd lose stock without a record. Wrapping the method in a transaction makes MySQL undo step 1 automatically. The four guarantees are called **ACID**: Atomic, Consistent, Isolated, Durable.

Normalization in one rule

Store each fact in exactly one place. `vehicles` stores `customer_id`, not the customer's name. If the name changes, you change one row in `customers` and every vehicle shows the new name automatically.

4. JPA and Hibernate: Java objects ↔ database rows

You write Java classes; Hibernate turns them into tables and writes the SQL for you. This translation is called **ORM** (Object-Relational Mapping).

- **JPA** (Jakarta Persistence API) is the *standard*: a set of annotations like `@Entity` and `@Id`.
- **Hibernate** is the *implementation* that actually does the work.
- **Spring Data JPA** sits on top and writes the boring database code (repositories) for you.

Entities

An entity is a Java class that maps to one table. One object = one row. Here's your `Vehicle`, annotated:

```
@Entity                                // "This class is a table"
@Table(name = "vehicles")              // ...named vehicles
public class Vehicle {

    @Id                                // Primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // MySQL
    AUTO_INCREMENT
    private Long id;

    @Column(nullable = false, unique = true) // NOT NULL + UNIQUE constraint
    private String licensePlate;           // column: license_plate (camelCase
    → snake_case)

    @ManyToOne(fetch = FetchType.LAZY)    // Many vehicles → one customer
    @JoinColumn(name = "customer_id", nullable = false) // The foreign key
    column
    private Customer customer;

    public Vehicle() {}                  // JPA requires an empty
    constructor
    // getters and setters...
}
```

Notice the field is a `Customer` object, not a `Long customerId`. In Java you work with objects; Hibernate stores only the id in the `customer_id` column and rebuilds the object when you read it.

Relationship annotations

Annotation	Placed on	Example in your code
<code>@ManyToOne</code> + <code>@JoinColumn</code>	The side that holds the FK	<code>Vehicle.customer</code> , <code>JobCard.vehicle</code> , <code>Payment.invoice</code>
<code>@OneToMany(mappedBy = "...")</code>	The other side (a <code>List</code>)	<code>Customer.vehicles (mappedBy = "customer")</code>
<code>@OneToOne</code> + <code>@JoinColumn</code>	One side of a 1:1	<code>Employee.user</code>
<code>@ManyToMany</code> + <code>@JoinTable</code>	Either side	<code>User.roles</code> through <code>user_roles</code>

`mappedBy` tells Hibernate: "the FK lives on the other class, don't create a second one here". The side with `@JoinColumn` is the **owning side**.

Cascade (`cascade = CascadeType.ALL` on `Customer.vehicles` and `Invoice.items`) means operations flow from parent to children: save an invoice and its items are saved too. **orphanRemoval = true** means removing an item from the list deletes its row.

Lazy vs eager loading

When you load a `Vehicle`, should Hibernate also load its `Customer`?

- **EAGER**: yes, immediately, with a join. Used on `User.roles` because security needs the roles on every login.
- **LAZY**: no. Hibernate puts a **proxy** (a placeholder object) in the field and only runs the query when you first call `vehicle.getCustomer().getName()`.

Lazy is the better default for performance, but it has a trap. If you return an entity straight from a controller, the JSON converter walks every field, hits a proxy, and either crashes or loops forever (`Vehicle` → `Customer` → `vehicles` → `Vehicle` → ...). That's exactly the bug in `InspectionController`, which returns `Inspection` entities directly. The fix is DTOs (section 5).

ddl-auto: who creates the tables?

`spring.jpa.hibernate.ddl-auto=update` in your `application.properties` tells Hibernate to compare your entities with the database at startup and add missing tables and columns.

Value	Effect	Use when
<code>create</code>	Drop everything, recreate. Deletes all data	Throwaway testing
<code>create-drop</code>	Like create, and drop again on shutdown	Automated tests
<code>update</code>	Add what's missing; never drops columns	Development (your current setting)
<code>validate</code>	Only check; fail if they differ	Production
<code>none</code>	Do nothing	Production with migration tools

`update` never removes or renames anything. If you rename a field, the old column stays in MySQL forever. Real projects use migration tools such as **Flyway** or **Liquibase**, which run versioned SQL scripts.

Repositories

A repository is the object you use to talk to one table. You write only an interface; Spring Data generates the class at startup.

```
public interface VehicleRepository extends JpaRepository<Vehicle, Long> {
    List<Vehicle> findByCustomerId(Long customerId);
}
```

`JpaRepository<Vehicle, Long>` (entity type, id type) gives you these for free: `save()`, `findById()`, `findAll()`, `count()`, `existsById()`, `deleteById()`.

Derived queries: SQL from method names

Spring reads the method name and writes the query.

Method in your code	SQL Spring generates
<code>findByUsername(String u)</code>	<code>SELECT * FROM users WHERE username = ?</code>
<code>existsByEmail(String e)</code>	<code>SELECT COUNT(*) > 0 FROM suppliers WHERE email = ?</code>
<code>findByCustomerId(Long id)</code>	<code>... WHERE customer_id = ?</code> (follows <code>customer.id</code>)

Method in your code	SQL Spring generates
<code>countByStatus(String s)</code>	<code>SELECT COUNT(*) FROM invoices WHERE status = ?</code>
<code>findByEmployeeIdOrderByPayMonthDescIdDesc(Long id)</code>	<code>... WHERE employee_id = ? ORDER BY pay_month DESC, id DESC</code>

When a name would get silly, write the query yourself with `@Query` in **JPQL**, which looks like SQL but uses class and field names instead of table and column names:

```
@Query("SELECT s FROM SparePart s WHERE s.quantityInStock <= s.reorderLevel")
List<SparePart> findLowStockParts();
```

`Optional<User>` (returned by `findById`) is a box that may be empty. You must decide what happens if it is: `.orElseThrow(() -> new RuntimeException("not found"))`.

5. The Spring Boot backend

Spring Boot is a framework: it runs a web server, receives HTTP requests, and calls *your* Java methods at the right moment. You write small classes with annotations; Spring wires them together.

Layered architecture

Every module is split into layers. Each layer has one job and only talks to the layer directly below it.

Layer	Annotation	Its one job	Must NOT
Controller	<code>@RestController</code>	Receive HTTP, read the URL/body, return a response with a status code	Contain business rules or talk to repositories
Service	<code>@Service</code>	Business rules: "is there enough stock?", "is this NIC taken?", calculations	Know about HTTP (no <code>ResponseEntity</code> here)
Repository	<code>@Repository</code> (interface)	Read and write one table	Contain business rules
Entity	<code>@Entity</code>	Describe a table's shape	Be sent to the browser directly

Layer	Annotation	Its one job	Must NOT
DTO	none (plain class)	The shape of data going in or out over HTTP	Be saved to the database

Why bother? If the rule "you can't dispense more parts than are in stock" lives in `InventoryService`, then every way of dispensing (the website, a future mobile app, a test) obeys it. If it lived in the frontend, anyone using Postman could skip it.

DTOs: Data Transfer Objects

A DTO is a simple class describing exactly what goes over the wire. Your project has two kinds:

- **Request DTO** (`CustomerRequest`): what the client may send. It has no `id`, because the client mustn't choose ids. It carries validation annotations.
- **Response DTO** (`CustomerResponse`): what the client receives. It leaves out secrets (`UserResponse` has no `password`) and flattens relationships (`VehicleResponse` has `customerName` instead of a whole `Customer` object).

The service converts between them with a small `mapToResponse(entity)` method. You'll see that pattern in every service you have.

DTOs solve three problems at once: they keep secrets out of JSON, they avoid the lazy-loading crash from section 4, and they let you change the database without breaking the frontend.

Dependency injection (DI)

This is the most important idea in Spring.

Your `CustomerController` needs a `CustomerService`. It never writes `new CustomerService()`. It just asks for one in its constructor:

```
@RestController
public class CustomerController {
    private final CustomerService customerService;

    public CustomerController(CustomerService customerService) { // Spring
        passes it in
        this.customerService = customerService;
    }
}
```

At startup Spring scans your packages, finds every class marked `@RestController`, `@Service`, `@Repository`, `@Component` or `@Configuration`, creates **one** object of each (called a **bean**), and passes them into the constructors that need them. The collection of beans is the **application context**.

Why this is good:

- **One shared instance:** every controller uses the same `CustomerService`.
- **Loose coupling:** a class says *what* it needs, not *how* to build it.
- **Testability:** in a test you can pass in a fake repository instead of a real database.

`@Bean` methods inside `@Configuration` classes are the other way to make beans. Your `SecurityConfig.passwordEncoder()` returns a `BCryptPasswordEncoder`, and Spring then injects it into `UserService` and `DataInitializer`.

The annotations you'll meet

Annotation	What it does	Example
<code>@SpringBootApplication</code>	The starting point; turns on scanning and auto-configuration	<code>ScannerPointApplication</code>
<code>@RestController</code>	Class handles HTTP and returns JSON	<code>CustomerController</code>
<code>@RequestMapping("/api/customers")</code>	URL prefix for every method in the class	
<code>@GetMapping</code> , <code>@PostMapping</code> , <code>@PutMapping</code> , <code>@PatchMapping</code> , <code>@DeleteMapping</code>	Method handles that HTTP method (+ optional extra path)	<code>@GetMapping("/{id}")</code>
<code>@PathVariable</code>	Reads <code>{id}</code> from the URL	<code>getEmployeeById(@PathVariable Long id)</code>
<code>@RequestParam</code>	Reads <code>?status=DONE</code>	<code>updateStatus(..., @RequestParam String status)</code>
<code>@RequestBody</code>	Converts the JSON body into a DTO	<code>createCustomer(@RequestBody CustomerRequest r)</code>
<code>@Valid</code>	Runs the DTO's validation annotations first	
<code>@PreAuthorize("hasRole('ADMIN')")</code>	Only these roles may call this method	<code>UserController.getAllUsers</code>

Annotation	What it does	Example
<code>@Transactional</code>	Run the whole method in one database transaction	<code>InvoiceService.createInvoice</code>
<code>@Component</code>	A general bean	<code>DataInitializer</code>

`ResponseEntity<T>` is the full HTTP response: body plus status code plus headers.

`ResponseEntity.ok(x)` = 200 with `x` as JSON. `new ResponseEntity<>(x, HttpStatus.CREATED)` = 201.

Startup: what happens when you press Run

1. `main()` in `ScannerPointApplication` calls `SpringApplication.run(...)`.
2. Spring reads `application.properties` (port, database URL, password).
3. It scans `ScannerPoint.example.ScannerPoint` and every sub-package for annotated classes, and builds the beans.
4. Hibernate connects to MySQL and runs `ddl-auto=update` against your entities.
5. Spring Security builds its filter chain from `SecurityConfig`.
6. `DataInitializer.run()` executes, because it implements `CommandLineRunner`: it inserts the five roles and the admin user if missing.
7. An embedded **Tomcat** web server starts listening on port 8081. You see `Started ScannerPointApplication in ... seconds`.

The life of one request inside Spring

1. Tomcat receives the bytes and hands them to Spring.
2. **Security filters** run first: CORS check, read the `Authorization` header, check URL rules (section 7).
3. The **DispatcherServlet** (Spring's traffic controller) finds which controller method matches the method + URL.
4. Jackson turns the JSON body into a DTO. `@Valid` checks it; on failure you get 400 and your method never runs.
5. `@PreAuthorize` checks the role.
6. Your controller method runs, calls the service, which calls the repository, which runs SQL.
7. The method returns a `ResponseEntity`; Jackson turns the DTO back into JSON.
8. If any exception escaped, Spring's error handling turns it into an error response (section 6).

6. Validation, exceptions and error handling

Never trust data from the client. Check it at the door (validation), refuse bad requests with a clear 4xx message, and make sure real crashes never leak internal details.

Bean Validation

Put constraint annotations on request DTO fields, then add `@Valid` before `@RequestBody` in the controller. Spring checks everything before your method runs.

Annotation	Rule	Used in your code on
<code>@NotNull</code>	Must be present	<code>VehicleRequest.customerId</code>
<code>@NotBlank</code>	Text must contain non-space characters	<code>CustomerRequest.name</code>
<code>@NotEmpty</code>	List must have at least one item	<code>InvoiceRequest.items</code>
<code>@Email</code>	Looks like an email	<code>RegisterRequest.email</code>
<code>@Size(min = 6)</code>	Length limits	<code>RegisterRequest.password</code>
<code>@Min(0)</code> , <code>@Positive</code>	Number limits	<code>SparePartRequest.price</code> , <code>PaymentRequest.amount</code>
<code>@Pattern(regexp = "...")</code>	Matches a regular expression	<code>EmployeeRequest.nic</code> (Sri Lankan NIC: 9 digits + V/X, or 12 digits)
<code>@Future</code> , <code>@PastOrPresent</code>	Date rules	<code>AppointmentRequest.appointmentDate</code>

If validation fails, Spring throws `MethodArgumentNotValidException` and replies 400. Because you set `server.error.include-binding-errors=always`, the response lists each failing field.

Validation checks the *shape* of one request. Business rules that need the database ("is this NIC already used?") belong in the service.

Exceptions

An exception is Java's way of saying "stop, something went wrong". `throw` raises it; it travels up through the methods that called it until something `catch`es it.

Your services use `throw new RuntimeException("Customer not found")`. Nobody catches it, so Spring's default handler turns it into **500 Internal Server Error**. The frontend can't tell "not found" from "the server is broken".

The fix: custom exceptions + a global handler

The standard pattern has two parts.

1. Exception classes that carry meaning:

```
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) { super(message); }
}
public class ConflictException extends RuntimeException { ... } //
duplicates
public class BadRequestException extends RuntimeException { ... } // broken
business rule
```

2. One `@RestControllerAdvice` class that catches them everywhere and picks the status:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<Map<String, String>> notFound(NotFoundException e)
    {
        return ResponseEntity.status(404).body(Map.of("message",
e.getMessage()));
    }

    @ExceptionHandler(ConflictException.class)
    public ResponseEntity<Map<String, String>> conflict(ConflictException e)
    {
        return ResponseEntity.status(409).body(Map.of("message",
e.getMessage()));
    }
}
```

Now the service just says `throw new NotFoundException("Customer not found")` and the client receives 404 with `{"message": "Customer not found"}`. Every error in the whole app has the same JSON shape, so the frontend needs only one function to display errors.

Defence in depth

Validate in three places, each for a different reason:

1. **Frontend**: instant feedback for the user (a red message under an empty field). Easily bypassed, so never the only check.
2. **Backend DTO + service**: the real gatekeeper.
3. **Database constraints** (`NOT NULL` , `UNIQUE` , foreign keys): the last line, catching anything the code missed.

7. Security

Security answers two separate questions on every request: **who are you?** (authentication) and **are you allowed to do this?** (authorization). Get the first wrong and you get 401; the second, 403.

Users, roles and passwords

Your `users` table holds accounts; `roles` holds `ROLE_ADMIN` , `ROLE_RECEPTIONIST` , `ROLE_MECHANIC` , `ROLE_STOREKEEPER` and `ROLE_USER` , created by `DataInitializer` . `user_roles` links them.

A user is **not** the same as an employee. `users` = login accounts. `employees` = HR records with salary and NIC. `Employee.user` optionally links the two.

Password hashing with BCrypt

Passwords are never stored as typed. `passwordEncoder.encode("admin123")` produces something like `$2a$10$N9qo8uL0ickgx2ZMRZoMy...`

- A **hash** is one-way: you can't turn it back into the password.
- To check a login, Spring hashes what the user typed and compares the two hashes.
- BCrypt adds a random **salt**, so two users with the same password get different hashes, and it's deliberately slow so attackers can't try billions of guesses.

The Spring Security filter chain

Before any controller runs, every request passes through a chain of **filters** configured in `SecurityConfig.filterChain()` :

```

http
  .csrf(csrf -> csrf.disable())
  .cors(Customizer.withDefaults())
  .authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()           // login,
register: open
    .requestMatchers("/api/inventory/**").hasAnyRole("ADMIN",
"STOREKEEPER")
    .anyRequest().authenticated()                         // everything
else: logged in
    .httpBasic(Customizer.withDefaults());                 // read the
Basic header

```

Rules are checked **top to bottom; the first match wins**. Note `hasRole("ADMIN")` automatically adds the `ROLE_` prefix, so it matches `ROLE_ADMIN` in the database.

Two layers of authorization

1. **URL rules** in `SecurityConfig` (above), checked in the filter chain.
2. **Method rules** with `@PreAuthorize("hasAnyRole('ADMIN', 'MECHANIC')")` on controller methods, checked later. Enabled by `@EnableMethodSecurity`.

A request must pass **both**. That's the cause of your dispense bug: the URL rule for `/api/inventory/**` allows only ADMIN and STOREKEEPER, so a mechanic is rejected before the method's `@PreAuthorize` (which would allow them) is ever reached. Pick one place for rules, ideally `@PreAuthorize`, and keep URL rules broad.

How login works today: HTTP Basic

With Basic auth there's no real "login". The client sends the username and password on **every** request:

```
Authorization: Basic YWRtaW46YWRtaW4xMjM=
```

That gibberish is just `admin:admin123` encoded in **Base64**. Base64 is *encoding*, not encryption: anyone can decode it. So:

1. Spring's `BasicAuthenticationFilter` decodes the header.
2. It calls `MyUserDetailsService.loadUserByUsername("admin")`, which loads the `User` from MySQL and wraps it in a `UserPrincipal`.
3. It checks the password with BCrypt.
4. If correct, it stores the user and their roles in the **SecurityContext** for this one request.

`UserPrincipal` is an adapter: Spring Security only understands its own `UserDetails` interface, so `UserPrincipal` translates your `User` entity into it (`getAuthorities()` returns the role names).

Your `POST /api/auth/login` only checks the password and returns a sentence. It gives the client nothing to keep, so the old frontend had to store the raw password in the browser.

Sessions vs tokens (JWT)

There are three common designs:

Approach	How it works	Trade-off
HTTP Basic (current)	Password sent with every request	Simplest. Password lives in the browser; only safe over HTTPS
Session cookie	Login once; server stores a session and gives the browser a cookie with a random id	Classic and safe. Server must remember sessions; needs CSRF protection
JWT (JSON Web Token)	Login once; server returns a signed token; client sends <code>Authorization: Bearer <token></code>	Stateless, popular with React apps. Logout and revoking tokens are harder

A JWT has three Base64 parts: header, payload (`{"sub": "admin", "roles": [...], "exp": ...}`) and a **signature**. The server signs with a secret key; if anyone edits the payload, the signature no longer matches. For a university project, either a session cookie or JWT is a solid upgrade from Basic.

CORS

Browsers enforce the **same-origin policy**: a page from `localhost:5173` may not read responses from `localhost:8081`, because a different port is a different **origin** (origin = scheme + host + port).

CORS (Cross-Origin Resource Sharing) is how the server says "I allow that origin". Your `CorsConfig` allows `http://localhost:5173` and `:3000` for the listed methods.

For anything other than a simple GET, the browser first sends a **preflight** `OPTIONS` request asking permission. If the method isn't on the list, the browser blocks the real request. That's why `PATCH` was missing until your last commit, and status updates silently failed. CORS is only enforced by **browsers**. Postman and curl ignore it entirely.

CSRF

Cross-Site Request Forgery: an evil website makes your browser send a request to your app, and the browser automatically attaches your cookies. CSRF protection adds a secret token that the evil site can't know.

You disabled it (`csrf.disable()`). That's acceptable **only** because Basic auth and JWT don't rely on cookies the browser sends automatically. If you switch to session cookies, turn it back on.

Security checklist for ScannerPoint

- Never commit passwords. Your MySQL password is in `application.properties` on GitHub. Use an environment variable: `spring.datasource.password=${DB_PASSWORD}`.
- Change or remove the seeded `admin` / `admin123` before anyone else can reach the app.
- Never trust the frontend to hide buttons as "security". The backend must check every request.
- Use HTTPS in production, or Basic auth and tokens can be read off the network.
- Return the same message for "wrong username" and "wrong password", so attackers can't discover which usernames exist.

8. Frontend fundamentals

Every website, however it's built, ends up as three languages in the browser: **HTML** for structure, **CSS** for looks, **JavaScript** for behaviour. React is a tool for producing them, not a replacement.

HTML: structure

HTML describes *what* is on the page using nested **tags**:

```
<form>
  <label for="name">Customer name</label>
  <input id="name" type="text" required />
  <button type="submit">Save</button>
</form>
<table>
  <tr><th>Name</th><th>Phone</th></tr>
  <tr><td>Nimal Perera</td><td>0771234567</td></tr>
</table>
```

Tags you'll use constantly: `div` (generic box), `h1 - h3` (headings), `p`, `a` (link), `button`, `form`, `input`, `select`, `table`, `ul / li` (lists), `nav`, `main`. Use the tag that means what you're showing (`button` for a button, not a clickable `div`); screen readers and keyboards depend on it.

CSS: presentation

CSS says how things look. A **selector** picks elements; **properties** style them.

```
.card { padding: 16px; border-radius: 8px; background: white; }
.btn-primary { background: #1e6fd9; color: white; }
.page { display: flex; gap: 24px; }          /* side-by-side layout */
.grid { display: grid; grid-template-columns: repeat(3, 1fr); }
@media (max-width: 600px) { .page { flex-direction: column; } } /* phones */
```

Key ideas: the **box model** (content → padding → border → margin), **Flexbox** for rows and columns, **Grid** for 2D layouts, **media queries** for different screen sizes ("responsive design"), and **CSS variables** (`--brand: #1e6fd9`) for a consistent theme.

JavaScript: behaviour

JavaScript is the only programming language browsers run. The modern features React code depends on:

```
const name = 'Nimal';           // const: can't be reassigned; let: can
const greet = (n) => `Hello ${n}`; // arrow function + template string
const customer = { id: 7, name, phone: '077...' };
const { id, phone } = customer; // destructuring: pull fields out
const copy = { ...customer, name: 'Sita' }; // spread: copy with a change
const names = customers.map(c => c.name);    // transform every item
const lowStock = parts.filter(p => p.isLowStock); // keep some items
const total = items.reduce((sum, i) => sum + i.amount, 0); // combine into
one
export default Customers; import Customers from './pages/Customers'; //
modules
```

Learn `map`, `filter`, destructuring and spread well. React code is mostly made of them.

The DOM

When the browser reads HTML, it builds a tree of objects in memory called the **DOM** (Document Object Model). What you see on screen is the DOM drawn. JavaScript can change the DOM (`document.querySelector('#name').value = 'x'`) and the screen updates.

Changing the DOM by hand gets messy fast in a big app: you have to remember to update every place that shows the customer count. That problem is why React exists.

Multi-page vs single-page apps

	Multi-page app (traditional)	Single-page app (SPA)
Click a link	Browser downloads a whole new HTML page	JavaScript swaps part of the page; no reload
Who builds the HTML	The server	The browser, using JavaScript
Data from server	HTML	JSON from an API
Examples	Old PHP sites, Spring with Thymeleaf	Gmail, ScannerPoint with React

ScannerPoint is an SPA. The server sends one nearly empty `index.html` plus a JavaScript bundle once; after that the app only fetches JSON from `/api/...`. This is why your backend has no HTML at all: it's a pure **API**.

9. React

React's core idea: **the UI is a function of state**. You describe what the screen should look like for the current data; when the data changes, React works out the minimal DOM changes and applies them.

Components

A component is a JavaScript function that returns what to show. Names start with a capital letter.

```
function CustomerRow({ customer }) {  
  return (  
    <tr>  
      <td>{customer.name}</td>  
      <td>{customer.phone}</td>  
    </tr>  
  );  
}
```

You build pages by nesting components: `App` → `AppShell` (sidebar + top bar) → `Customers` page → `DataTable` → `CustomerRow`. Small reusable pieces (`Button`, `Input`, `Card`) are written once and used everywhere.

JSX

The HTML-looking code inside JavaScript is **JSX**. A build tool converts it to plain JavaScript function calls. Differences from HTML:

- `className` instead of `class`; `htmlFor` instead of `for`.
- `{ }` drops into JavaScript: `<td>{customer.name}</td>`, `<p>{items.length} items</p>`.
- A component returns one root element (wrap siblings in `<>...</>`, a **fragment**).
- Lists: `{customers.map(c => <CustomerRow key={c.id} customer={c} />)}`. The `key` lets React track which row is which.
- Conditions: `{loading && <p>Loading...</p>}` or `{error ? <Notice>{error}</Notice> : <Table/>}`.

Props: data passed down

Props are a component's inputs, passed like HTML attributes: `<CustomerRow customer={c} />`. The child receives them as an object and must treat them as **read-only**. Data flows **down** the tree; to send something **up** ("this row was clicked"), the parent passes a function as a prop: `<Button onClick={handleSave}>`.

State: data that changes

State is data a component remembers between renders. Changing it with the setter makes React re-render.

```
import { useState } from 'react';

function AddCustomerForm() {
  const [name, setName] = useState(''); // [current value, setter],
  initial ''

  return (
    <input value={name} onChange={e => setName(e.target.value)} />
  );
}
```

This is a **controlled input**: React state is the single source of truth, and the input just displays it. Never change state directly (`name = 'x'` or `list.push(x)`); always call the setter with a new value (`setList([...list, x])`).

Hooks

Functions starting with `use` are **hooks**. Rules: call them only at the top level of a component, never inside `if` or loops.

Hook	Purpose
<code>useState</code>	Remember a value; re-render when it changes
<code>useEffect</code>	Run side effects after rendering: fetching data, timers
<code>useContext</code>	Read shared data without passing props through every level (the logged-in user)
<code>useCallback</code> , <code>useMemo</code>	Keep the same function or computed value between renders (performance)
Custom hooks	Your own reusable logic, e.g. the old <code>useResource(path)</code> that fetched data

`useEffect` and loading data

```
const [customers, setCustomers] = useState([]);
const [error, setError] = useState('');

useEffect(() => {
  API.get('/customers')
    .then(res => setCustomers(res.data))
    .catch(err => setError('Could not load customers'));
}, []); // [] = run once, when the component first appears
```

The second argument is the **dependency array**: the effect re-runs whenever a value in it changes. `[customerId]` means "re-fetch when the selected customer changes". Forget it entirely and the effect runs after every render, which can loop forever.

Context: app-wide data

The logged-in user is needed on almost every page. Instead of passing it as a prop through ten levels, put it in a **Context**: `AuthProvider` wraps the whole app, and any component calls `useAuth()` to get `user` , `login` and `logout` . Your deleted `AuthContext.jsx` did

exactly this.

Routing with React Router

An SPA still needs URLs, so users can bookmark `/customers` and use the Back button. React Router maps URL paths to components without reloading:

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route element={<RequireAuth><AppShell /></RequireAuth>}>
    <Route path="/customers" element={<Customers />} />
    <Route path="/job-cards" element={<JobCards />} />
  </Route>
</Routes>
```

`<Link to="/customers">` changes the URL without a reload; `useNavigate()` does it from code (after login). A **protected route** like `RequireAuth` redirects to `/login` when nobody is signed in. Remember this only hides screens. Real protection is the backend's job.

Vite and npm

- **Node.js** runs JavaScript outside the browser. You need it for the tools, not for the app itself.
- **npm** is the package manager: `npm install` downloads everything listed in `package.json` into `node_modules/` (never commit that folder).
- **Vite** is the dev server and build tool. `npm run dev` serves the app at `localhost:5173`, converts JSX, and refreshes the browser instantly when you save (**hot module replacement**). `npm run build` creates an optimized `dist/` folder of plain HTML, CSS and JS that any web server can host.

Typical folder layout

```

scannerpoint-web/
├─ index.html           the one HTML page; loads src/main.jsx
├─ package.json         dependencies and scripts
├─ vite.config.js
├─ .env                 VITE_API_BASE_URL=http://localhost:8081/api
├─ src/
│   ├─ main.jsx         starts React, wraps App in Router + AuthProvider
│   ├─ App.jsx          the routes
│   ├─ api/             axios setup, one file of calls per module
│   ├─ context/         AuthContext
│   ├─ components/      reusable pieces: layout/, ui/
│   ├─ pages/           one component per screen: Customers.jsx,
│   JobCards.jsx...
│   └─ styles/          CSS

```

10. Connecting the frontend to the backend

The React app talks to Spring with HTTP requests made from JavaScript. Three things have to line up: the URL, the JSON shape, and the login header.

Asynchronous code: promises and async/await

A network request takes time (milliseconds to seconds). JavaScript doesn't stop and wait; it keeps the page responsive and comes back when the answer arrives. A **Promise** is an object representing "a value that will arrive later", which ends up either **fulfilled** (success) or **rejected** (error).

Two ways to write the same thing:

```
// .then / .catch
API.get('/customers')
  .then(res => setCustomers(res.data))
  .catch(err => setError(err.message));

// async / await: reads top to bottom, use try/catch for errors
async function loadCustomers() {
  try {
    const res = await API.get('/customers');
    setCustomers(res.data);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
}
```

`await` can only be used inside an `async` function. It pauses *that function* only, not the whole page.

axios

Browsers have a built-in `fetch()`, but **axios** is friendlier: it converts JSON automatically, rejects on 4xx/5xx, and supports **interceptors**. You create one configured instance and use it everywhere:

```
// src/api/axios.js
const API = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL, // http://localhost:8081/api
  headers: { 'Content-Type': 'application/json' },
});

// Request interceptor: runs before EVERY request → attach the login
API.interceptors.request.use(config => {
  const token = getSavedToken();
  if (token) config.headers.Authorization = `Bearer ${token}`; // or `Basic
  ...` today
  return config;
});

// Response interceptor: runs on EVERY response → handle 401 in one place
API.interceptors.response.use(
  res => res,
  err => { if (err.response?.status === 401) logoutAndGoToLogin(); return
  Promise.reject(err); }
);
```

Then calling the backend is one line: `API.post('/customers', { name, phone, email })`. The response object has `res.data` (the parsed JSON), `res.status`, and `res.headers`. On failure, `err.response.data` holds the backend's error JSON, e.g. `{"message": "Customer not found"}`.

Keep API calls in one place

Instead of scattering URLs through pages, put each module's calls in a file:

```
// src/api/customers.js
export const getCustomers = () => API.get('/customers').then(r => r.data);
export const createCustomer = (body) => API.post('/customers', body).then(r => r.data);
export const getVehicles = (id) =>
  API.get(`/vehicles/customer/${id}`).then(r => r.data);
```

If a backend URL changes, you fix one line.

The contract: frontend and backend must agree

The JSON keys the frontend sends must match the request DTO's field names exactly, letter for letter. Jackson matches `licensePlate` in JSON to `setLicensePlate()` in `VehicleRequest`. Send `license_plate` and the field silently stays `null`, then

`@NotBlank` rejects it.

Backend says	Frontend must do
<code>@PostMapping + @RequestBody VehicleRequest</code>	<code>API.post('/vehicles', { customerId, licensePlate, make, model })</code>
<code>@PatchMapping("/{id}/status") + @RequestParam String status</code>	<code>API.patch(`/appointments/\${id}/status`, null, { params: { status } })</code>
<code>@GetMapping("/customer/{customerId}")</code>	<code>API.get(`/vehicles/customer/\${customerId}`)</code>
<code>LocalDateTime appointmentDate</code>	Send ISO text: <code>"2026-10-15T09:30:00"</code>

Teams often document this contract with **OpenAPI/Swagger**: add the `springdoc-openapi` dependency and you get an interactive page listing every endpoint and its JSON.

Environment variables

The backend address differs between your laptop and a real server, so it isn't hard-coded. Vite reads `.env` files; only variables starting with `VITE_` reach the browser code, via `import.meta.env.VITE_API_BASE_URL`. Use `.env.development` and `.env.production` for different values.

Anything in frontend code is visible to every user (open DevTools → Sources). **Never put secrets in the frontend.**

The three UI states of every request

Every screen that loads data must handle all three:

1. **Loading**: show a spinner or "Loading..."; disable the Save button so it can't be double-clicked.
2. **Error**: show the backend's message ("NIC already exists") near the form.
3. **Success**: show the data, or a confirmation, and refresh the list.

Debugging the connection

The browser's **DevTools** (right-click → Inspect) is your best friend:

- **Network tab**: every request, its status code, headers, request body and response. Start here for any "it doesn't work".
- **Console tab**: JavaScript errors and your `console.log` output. A CORS failure shows up here in red.

And test the backend alone, without React, using **Postman**, **Bruno** or `curl`:

```
curl -u admin:admin123 http://localhost:8081/api/customers
```

If curl works but the browser doesn't, the problem is CORS or the frontend. If curl fails too, it's the backend.

Symptom	Likely cause
"Network Error", no status, CORS message in console	Backend not running, wrong port, or origin/method missing from <code>CorsConfig</code>
401	Missing or wrong <code>Authorization</code> header
403	Logged in, but the role isn't allowed (check <code>SecurityConfig</code> and <code>@PreAuthorize</code>)
400	JSON keys don't match the DTO, or validation failed. Read the response body
404	Wrong URL, or wrong method (POST to a GET-only path usually gives 405)
500	Exception in Java. Read the stack trace in the backend console

11. Build tools and configuration

Build tools download the libraries your code depends on, compile it, and run it. The backend uses **Maven**; the frontend uses **npm**. They do the same job for different languages.

	Backend	Frontend
Tool	Maven (<code>./mvnw</code>)	npm
Dependency list	<code>pom.xml</code>	<code>package.json</code>
Where libraries go	<code>~/.m2/repository</code> (shared)	<code>node_modules/</code> (per project)
Exact versions locked in	<code>pom.xml</code> itself	<code>package-lock.json</code>
Run in development	<code>./mvnw spring-boot:run</code> (or IntelliJ's Run button)	<code>npm run dev</code>

	Backend	Frontend
Build for production	<code>./mvnw clean package → target/ScannerPoint-*.jar</code>	<code>npm run build → dist/</code>
Output folder (never commit)	<code>target/</code>	<code>node_modules/</code> , <code>dist/</code>

pom.xml

The **Project Object Model** describes the backend. The important parts of yours:

- `<parent>spring-boot-starter-parent</parent>` : inherits sensible versions for every Spring library, so you rarely write version numbers.
- `<java.version>21</java.version>` : the Java version.
- `<dependencies>` : the **starters** you use. A starter is a bundle of libraries for one job.

Starter	What it brings
<code>spring-boot-starter-webmvc</code>	Embedded Tomcat, controllers, Jackson for JSON
<code>spring-boot-starter-data-jpa</code>	Hibernate and Spring Data repositories
<code>spring-boot-starter-security</code>	The filter chain, BCrypt, <code>@PreAuthorize</code>
<code>spring-boot-starter-validation</code>	<code>@NotBlank</code> , <code>@Email</code> , <code>@Valid</code> ...
<code>mysql-connector-j</code>	The JDBC driver : the low-level code that speaks MySQL's network protocol
<code>*-test</code> starters	JUnit and testing helpers, used only by <code>src/test</code>

After editing `pom.xml` , reload Maven in IntelliJ (the small elephant icon) so it downloads the new libraries.

`mvnw` is the **Maven Wrapper**: a script that downloads the right Maven version for you, so teammates don't need Maven installed.

application.properties

Spring's settings file. Every line in yours, explained:

Property	Meaning
<code>spring.datasource.url=jdbc:mysql://localhost:3306/scannerpoint_db?createDatabaseIfNotExist=true...</code>	Where MySQL is, which database, and to create it if it's missing
<code>spring.datasource.username / password</code>	MySQL login. Should come from an environment variable, not be committed
<code>spring.jpa.hibernate.ddl-auto=update</code>	Let Hibernate update tables at startup (section 4)
<code>spring.jpa.show-sql=true</code>	Print every SQL query to the console. Great for learning
<code>server.error.include-message=always</code>	Put exception messages in error responses
<code>server.port=8081</code>	The backend's port

Profiles let you keep different settings per environment: `application-dev.properties` and `application-prod.properties`, chosen with `spring.profiles.active=dev`.

Backend package layout

Java packages mirror folders. The package is named `ScannerPoint.example.ScannerPoint`; by convention package names are all lowercase and start with a reversed domain (e.g. `lk.sliit.scannerpoint`), but renaming now is optional.

```

backend/
├─ pom.xml, mvnw
└─ src/
    ├─ main/java/ScannerPoint/example/ScannerPoint/
    │   ├─ ScannerPointApplication.java    main()
    │   ├─ config/                        SecurityConfig, CorsConfig, DataInitializer
    │   ├─ user/                          entity, repository, service, controller, dto, security
    │   ├─ customer/                      same layers
    │   ├─ repair/                        appointments, job cards, inspections
    │   ├─ inventory/                     parts, suppliers, stock
    │   ├─ billing/                       invoices, payments, reports
    │   └─ employee/                      employees, payroll, staff report
    ├─ main/resources/application.properties
    └─ test/java/...                      tests

```

This is **package-by-feature**: each business area is self-contained, so team members can work on different modules without stepping on each other.

12. Git and team workflow

Git records snapshots of your project so you can go back in time, and lets a team work in parallel without overwriting each other. GitHub hosts a shared copy of the repository.

Core concepts

- **Repository (repo)**: the project folder plus its full history (hidden in `.git/`).
- **Commit**: a saved snapshot with a message and an author. Each has an id like `1b389e3`.
- **Branch**: a movable label pointing at a line of commits. `main` is the shared, working version. Your team has branches like `Sewmini_billings` and `Tevindu_Inventory`.
- **Remote**: the copy on GitHub, named `origin`.
- **Merge**: combining one branch's commits into another.
- **Pull request (PR)**: a GitHub page asking "please merge my branch into main", where teammates review the changes first.
- **Merge conflict**: two branches changed the same lines; Git asks a human to choose.

The three areas

```
working folder --git add--> staging area --git commit--> local history -
-git push--> GitHub
<--git pull--
```

Daily commands

```
git status                # what changed?
git pull                  # get teammates' latest work
git switch -c feature/customer-edit # new branch for one task
git add src/main/java/.../CustomerService.java # stage specific files
git commit -m "Add update endpoint for customers"
git push -u origin feature/customer-edit
# then open a pull request on GitHub

git log --oneline --graph  # history
git diff                  # unstaged changes, line by line
```

Rules that would have prevented the deleted frontend

Your PR #17 merged a commit called "redo" that deleted 64 frontend files. Because Git keeps history, nothing is truly lost: `git show f94b498` still has every file. These habits stop it happening by accident:

1. **One branch per task**, named for the task: `feature/vehicle-crud`, `fix/dispense-permission`.
2. **Small commits with clear messages** that say what and why: "Allow mechanics to dispense parts", not "redo" or "done".
3. **Read the PR's "Files changed" tab before merging.** A PR deleting 4,581 lines should make someone stop.
4. **Require one teammate's approval** before merging to `main` (GitHub → Settings → Branches → branch protection).
5. **Pull before you start, and before you push.**
6. **Set your identity once** so commits show your name, not "Your Name": `git config --global user.name "Your Real Name" and user.email`.

.gitignore

Lists files Git should never track: build output (`target/`, `node_modules/`, `dist/`), IDE settings (`.idea/`), OS junk (`.DS_Store`), and secrets (`.env.local`). Adding a file to `.gitignore` doesn't remove it if it was **already** committed; for that, run `git rm -r --cached .idea` once. That's why `.idea/` is still in your repo.

Undoing things safely

Situation	Command
Discard changes to one file you haven't committed	<code>git restore path/to/file</code>
Unstage a file	<code>git restore --staged path/to/file</code>
Undo a commit already pushed (creates a new "reverse" commit)	<code>git revert <commit-id></code>
Get one old file back	<code>git restore --source f94b498 -- path/to/file</code>

Avoid `git push --force` and `git reset --hard` on shared branches; they rewrite history your teammates already have.

13. One feature end to end: adding a customer

This traces a single click through every layer, using your real backend code and the frontend you'll rebuild. If you can explain each step here without looking, you understand full-stack development.

Step 1 — React form (browser). The receptionist types into a controlled form and clicks Save.

```
const [form, setForm] = useState({ name: '', phone: '', email: '', address: '' });

async function handleSubmit(e) {
  e.preventDefault(); // stop the browser's old-style full
page submit
  setSaving(true);
  try {
    const created = await createCustomer(form); // from
src/api/customers.js
    setCustomers(list => [...list, created]); // show it without re-
fetching
  } catch (err) {
    setError(err.response?.data?.message ?? 'Could not save');
  } finally {
    setSaving(false);
  }
}
```

Step 2 — axios builds the request. `createCustomer` calls `API.post('/customers', form)`. The base URL makes it `http://localhost:8081/api/customers`; `axios` turns `form` into JSON, and the request interceptor adds the `Authorization` header.

Step 3 — CORS preflight. Different origin plus a JSON body, so the browser first sends `OPTIONS /api/customers`. `CorsConfig` replies that `localhost:5173` may `POST`. The browser then sends the real request.

- ☐ **Step 4 — Security filters.** `BasicAuthenticationFilter` decodes the header, `MyUserDetailsService` loads the user from `users`, `BCrypt` checks the password. The URL rule `anyRequest().authenticated()` passes.

Step 5 — Routing. The `DispatcherServlet` matches `POST + /api/customers` to `CustomerController.createCustomer`.

Step 6 — Method security and validation. `@PreAuthorize("hasAnyRole('ADMIN', 'RECEPTIONIST')")` passes for a receptionist. Jackson fills a `CustomerRequest` from the JSON; `@Valid` checks `@NotBlank` `name`, `@NotBlank` `phone` and `@Email` `email`. A blank name would stop here with 400.

Step 7 — Controller → service.

```
@PostMapping
public ResponseEntity<CustomerResponse> createCustomer(@Valid @RequestBody
CustomerRequest request) {
    return new ResponseEntity<>(customerService.createCustomer(request),
HttpStatus.CREATED);
}
```

Step 8 — Service: DTO → entity → save.

```
Customer customer = new Customer(request.getName(), request.getEmail(),
request.getPhone(), request.getAddress());
Customer saved = customerRepository.save(customer);
return mapToResponse(saved);
```

Step 9 — Hibernate → MySQL. `save()` sees the entity has no id, so it's new. Hibernate generates and runs (and prints, thanks to `show-sql`):

```
insert into customers (address, email, name, phone) values (?, ?, ?, ?)
```

MySQL checks the constraints (`name` not null, `email` unique), writes the row, and returns the new `AUTO_INCREMENT` id, say `7`. Hibernate puts `7` into `saved.id`.

Step 10 — Back out through the layers. `mapToResponse` copies the entity into a `CustomerResponse`. The controller wraps it with status 201. Jackson calls the getters and writes JSON:

```
{"id": 7, "name": "Nimal Perera", "email": "nimal@mail.com", "phone":
"0771234567", "address": "Kandy"}
```

Step 11 — React updates. `await` resolves with the new customer (`createCustomer` already unwrapped `res.data`). `setCustomers` adds it to state, React re-renders, and a new row appears in the table. Total time: usually well under a second.

What if it goes wrong?

Where it fails	What the user should see	Status
Backend not running	"Cannot reach the server"	none (network error)

Where it fails	What the user should see	Status
Saved login is wrong or expired	Sent back to the login page	401
A mechanic tries it	"You don't have permission" (better: hide the button)	403
Name left blank	"Name is required" under the field	400
Email already used	"A customer with this email already exists"	409 (today: 500, until section 6's handler exists)

Your turn

Trace `POST /api/inventory/dispense` the same way on paper, step by step. You'll find exactly where a mechanic gets rejected (step 4, the URL rule) and why `@Transactional` matters (step 8, three writes).

14. Learning roadmap and glossary

Learn in the order data flows: database → backend → HTTP → frontend. Do each step hands-on with ScannerPoint before moving on.

Roadmap

- ☐ **SQL basics.** Open MySQL Workbench, connect to `scannerpoint_db`, and run `SELECT`, `INSERT`, `UPDATE` and a `JOIN` on `vehicles` + `customers`.
- ☐ **Java essentials.** Classes, constructors, interfaces, `List`, `Optional`, streams (`.stream().map(...).collect(...)`), exceptions.
- ☐ **Run the backend** from IntelliJ. Read the startup log and find the SQL that `ddl-auto` ran.
- ☐ **Call the API with Postman or curl.** Log in as `admin`, create a customer, a vehicle and an appointment. Trigger a 400, a 401 and a 403 on purpose.
- ☐ **Read one module top to bottom:** `customer` (smallest), then `repair`. For each endpoint, say which DTO, service method and repository method it uses.
- ☐ **Add one feature yourself:** `PUT /api/customers/{id}` and `DELETE /api/customers/{id}`.
- ☐ **Add the global exception handler** from section 6 and watch 500s become 404s and 409s.

- ☐ **HTML, CSS and modern JavaScript.** Build a plain HTML page that fetches `/api/customers` with `fetch()` and prints them.
- ☐ **React basics:** components, props, `useState`, `useEffect`, lists with `key`. The official tutorial at react.dev is excellent.
- ☐ **Rebuild the frontend** step by step: Vite project → axios instance → login page → customers page → router and layout → remaining modules.
- ☐ **Upgrade login** to a session cookie or JWT, and add `GET /api/auth/me`.
- ☐ **Tests:** one JUnit test for a service, one `@WebMvcTest` for a controller.

Glossary

Term	Meaning
API	A set of URLs a program exposes for other programs to call. Your backend is a REST API
Authentication	Proving who you are (login)
Authorization	Checking what you're allowed to do (roles)
Bean	An object Spring creates and manages, then injects where needed
CORS	Browser rule plus server permission for cross-origin requests
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object: the JSON shape for a request or response
Dependency injection	Spring hands each class the objects it needs, via the constructor
DOM	The browser's in-memory tree of the page
Endpoint	One method + URL, e.g. <code>POST /api/customers</code>
Entity	A Java class mapped to a database table
Foreign key	A column holding another table's id, linking rows
Hook	A React function starting with <code>use</code> (<code>useState</code> , <code>useEffect</code>)
Hibernate	The ORM library that turns entities into SQL
JPA	The Java standard for ORM annotations

Term	Meaning
JSON	The text format for data sent between frontend and backend
JSX	HTML-like syntax inside React JavaScript
JWT	A signed token proving login, sent as <code>Bearer</code>
Lazy loading	Loading a related object only when first used
ORM	Object-Relational Mapping: objects ↔ table rows
Primary key	The column that uniquely identifies a row (<code>id</code>)
Promise	A JavaScript value that arrives later (network results)
Props	Inputs passed from a parent React component to a child
Repository	Spring Data interface for reading and writing one table
REST	Conventions for organizing an API around resources and HTTP methods
Service	The layer holding business rules
SPA	Single-page app: one HTML page, JavaScript swaps the content
State	Data a React component remembers; changing it re-renders
Status code	The 3-digit result of an HTTP request (200, 404...)
Transaction	A group of database changes that all succeed or all fail